



Questões Práticas

Instruções: resolva as questões utilizando a linguagem Java. As soluções devem utilizar os conceitos de Orientação a Objetos trabalhados em sala de aula, como classes, atributos, métodos, construtores e encapsulamento. Também devem ser utilizados, quando solicitado, recursos como `List`, `ArrayList`, `Set`, `HashSet`, `Map`, `HashMap`, `Generics`, métodos da classe `String`, tratamento de exceções e geração de relatórios com `StringBuilder`.

1. Sistema Genérico de Repositório de Livros

Crie um sistema em Java para gerenciar livros de uma biblioteca utilizando `Generics` e coleções.

O sistema deve possuir uma classe genérica chamada `Repositorio<T>`, responsável por armazenar objetos de qualquer tipo em uma coleção.

A classe `Repositorio<T>` deve possuir, no mínimo, os seguintes métodos:

```
public void adicionar(T objeto)
public List<T> listar()
```

Não deve ser permitido adicionar objetos nulos ao repositório. Caso isso aconteça, deve ser lançada uma exceção do tipo `IllegalArgumentException`.

Em seguida, crie uma classe chamada `Livro`, contendo:

```
private String isbn;
private String titulo;
private String autor;
```

O ISBN, o título e o autor não podem ser nulos ou vazios. O título e o autor devem ser armazenados sem espaços no início ou no fim.

Além disso, crie uma classe chamada `Biblioteca`, que utilize:

```
private Repositorio<Livro> repositorio;
private Map<String, Livro> livrosPorIsbn;
```

A classe `Biblioteca` deve permitir:

- Cadastrar livros;
- Impedir cadastro de livro com ISBN repetido;
- Buscar livro pelo ISBN;
- Listar todos os livros cadastrados;

- e) Exibir mensagem adequada quando um livro não for encontrado.

No método `main`, cadastre pelo menos cinco livros, tente cadastrar um livro repetido e realize buscas válidas e inválidas.

Requisitos:

- a) Criar a classe genérica `Repositorio<T>`;
- b) Utilizar `List<T>` e `ArrayList<T>`;
- c) Criar a classe `Livro` com encapsulamento;
- d) Criar a classe `Biblioteca`;
- e) Utilizar `Map<String, Livro>` e `HashMap`;
- f) Validar dados com `IllegalArgumentException`;
- g) Impedir ISBN repetido;
- h) Utilizar `try-catch` nos testes do `main`.

2. Cadastro e Padronização de Funcionários

Crie um sistema em Java para cadastrar funcionários de uma empresa, aplicando manipulação de `String`.

O sistema deve possuir uma classe chamada `Funcionario`, contendo:

```
private String nome;  
private String cpf;  
private String email;  
private String departamento;
```

O construtor da classe deve receber todos os dados e realizar as seguintes validações e padronizações:

- a) O nome não pode ser nulo ou vazio;
- b) O nome deve ser armazenado sem espaços no início ou no fim;
- c) O CPF pode ser recebido com pontos e traço, mas deve ser armazenado apenas com números;
- d) Após a remoção dos caracteres especiais, o CPF deve possuir exatamente 11 caracteres;
- e) O e-mail deve conter o caractere `@`;
- f) O e-mail deve ser armazenado em letras minúsculas;
- g) O departamento não pode ser vazio e deve ser armazenado em letras maiúsculas.

Além disso, implemente o método:

```
public String gerarLogin()
```

Esse método deve gerar um login no seguinte formato:

```
primeiroNome.departamento
```

Por exemplo:

```
Nome: Joao da Silva
Departamento: tecnologia

Login gerado:
joao.TECNOLOGIA
```

Para obter o primeiro nome, utilize `split()`.

Também crie um método chamado `gerarRelatorio()`, utilizando `StringBuilder`, para retornar os dados formatados do funcionário.

No método `main`, crie uma lista de funcionários utilizando `List<Funcionario>`. Tente cadastrar funcionários válidos e inválidos. Os funcionários inválidos não devem ser adicionados à lista.

Requisitos:

- a) Criar a classe `Funcionario`;
- b) Utilizar atributos privados;
- c) Validar todos os dados no construtor ou em métodos auxiliares;
- d) Utilizar `trim()`, `replace()`, `contains()`, `toLowerCase()`, `toUpperCase()` e `split()`;
- e) Criar o método `gerarLogin()`;
- f) Criar o método `gerarRelatorio()` com `StringBuilder`;
- g) Utilizar `List<Funcionario>` e `ArrayList`;
- h) Tratar erros com `try-catch`.

3. Sistema de Gerenciamento de Turma com Processamento de Texto

Crie um sistema completo para gerenciar uma turma de alunos a partir de linhas de texto.

Cada aluno será informado no seguinte formato:

```
Nome;Matricula;Nota
```

Exemplo:

```
Maria Silva;9383;8.5
Joao Pedro;9321;7.0
Ana Clara;9210;9.5
Carlos Lima;9100;4.0
Maria Silva;9383;8.5
```

O sistema deve possuir uma classe chamada `Aluno`, contendo:

```
private String nome;  
private String matricula;  
private double nota;
```

Também deve possuir uma classe chamada **Turma**, responsável por armazenar e gerenciar os alunos.

A classe **Turma** deve utilizar:

```
private List<Aluno> alunos;  
private Map<String, Aluno> alunosPorMatricula;  
private Set<String> matriculasCadastradas;
```

O sistema deve:

- a) Ler cada linha de texto;
- b) Separar os dados utilizando `split(";")`;
- c) Criar objetos da classe **Aluno**;
- d) Impedir o cadastro de matrícula repetida;
- e) Validar nome vazio;
- f) Validar matrícula vazia;
- g) Validar nota fora do intervalo de 0 a 10;
- h) Calcular a média geral da turma;
- i) Identificar o aluno com maior nota;
- j) Identificar o aluno com menor nota;
- k) Gerar um relatório final usando **StringBuilder**.

Caso uma linha esteja inválida, o sistema deve tratar o erro com **try-catch**, exibir uma mensagem informando o problema e continuar processando as próximas linhas.

O relatório final deve seguir um formato semelhante ao exemplo abaixo:

```
RELATORIO DA TURMA  
-----  
  
Quantidade de alunos cadastrados: 4  
Media geral: 7.25  
  
Maior nota:  
9210 - Ana Clara - 9.5  
  
Menor nota:  
9100 - Carlos Lima - 4.0  
  
ALUNOS CADASTRADOS:  
9383 - Maria Silva - 8.5  
9321 - Joao Pedro - 7.0
```

Requisitos:

- a) Criar as classes `Aluno` e `Turma`;
- b) Utilizar encapsulamento;
- c) Utilizar `List<Aluno>` e `ArrayList`;
- d) Utilizar `Map<String, Aluno>` e `HashMap`;
- e) Utilizar `Set<String>` e `HashSet`;
- f) Utilizar `split(";")` para processar as linhas;
- g) Utilizar `trim()` para limpar os dados;
- h) Utilizar `IllegalArgumentException`;
- i) Tratar erros com `try-catch`;
- j) Impedir matrícula repetida;
- k) Calcular média, maior nota e menor nota;
- l) Gerar relatório com `StringBuilder`.